

# 作业 HW5-查找 实验报告

姓名：毛星博 学号：2351520 日期：2024.12.10

## 1. 涉及数据结构和相关背景

### 1.1 静态查找表

静态查找表是指查找过程中，查找表是静态的，即表中的记录数量和内容在查找操作期间是固定不变的。常见的静态查找表的查找方法包括顺序查找、二分查找和分块查找。

#### 1.1.1 顺序查找

顺序查找是最简单的一种查找方法。它从查找表的第一个元素开始，依次与关键字进行比较，直到找到关键字相等的元素或查找完所有元素。

**时间复杂度：**

- 最好情况： $O(1)$ （第一个元素即为所查元素）
- 最坏情况： $O(n)$ （所有元素都比较过）
- 平均情况： $O(n/2) \approx O(n)$

#### 1.1.2 二分查找

二分查找适用于已排序的查找表。查找过程是从表的中间元素开始，如果中间元素的关键字正好等于查找关键字，则查找成功；如果中间元素的关键字大于查找关键字，则在中间元素的左半部分继续查找；如果中间元素的关键字小于查找关键字，则在中间元素的右半部分继续查找。

**时间复杂度：**

- 最好情况： $O(1)$ （中间元素即为所查元素）
- 最坏情况： $O(\log n)$
- 平均情况： $O(\log n)$

#### 1.1.3 分块查找

分块查找将查找表分成若干块，每块的大小不一定相同，但通常相同。每块中记录的关键字按有序顺序排列，同时每块建立一个索引表，索引表中的每个元素包含该块的最大关键字和该块的起始地址。

**时间复杂度：**

- 最好情况： $O(1)$ （目标关键字在索引表的第一个块中）
- 最坏情况： $O(\sqrt{n})$ （目标关键字在索引表的最后一个块中且块内顺序查找也需要最大次数）
- 平均情况： $O(\sqrt{n})$

### 1.2. 动态查找表

动态查找表是指查找过程中，查找表是动态的，即表中的记录数量和内容在查找操作期间可能会发生变化。常见的动态查找表的数据结构包括二叉排序树和平衡二叉树等。

#### 1.2.1 二叉排序树 (BST)

二叉排序树是一种动态查找表，它或者是一棵空树，或者是具有以下性质的二叉树：

- 若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值。
- 若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值。
- 它的左、右子树也分别为二叉排序树。

**基本操作：**

- 插入：找到要插入的位置，插入新结点。
- 删除：找到要删除的结点，根据结点情况（叶子结点、单分支结点、双分支结点）进行删除。
- 查找：从根结点开始，根据关键字大小递归查找。

**时间复杂度：**

- 最好情况： $O(\log n)$
- 最坏情况： $O(n)$ （树退化成链表）
- 平均情况： $O(\log n)$

### 1.2.2 平衡二叉树 (AVL 树)

平衡二叉树是一种特殊的二叉排序树，其中每个结点的左子树和右子树的高度之差的绝对值不超过 1。平衡二叉树能够保证查找操作的时间复杂度为  $O(\log n)$ 。平衡二叉树通过平衡因子来判断树是否平衡。平衡因子是指结点的左子树高度减去右子树高度。

**平衡因子：**

- 如果平衡因子为 0，表示左右子树高度相同，树平衡。
- 如果平衡因子为 -1，表示右子树高度比左子树高 1，树左偏。
- 如果平衡因子为 1，表示左子树高度比右子树高 1，树右偏。
- 如果平衡因子小于 -1 或大于 1，表示树不平衡，需要进行旋转操作来恢复平衡。

**基本操作：**

- 插入：插入结点后，若破坏了平衡，需要进行旋转操作（单左旋、单右旋、左右旋、右左旋）来恢复平衡。
- 删除：删除结点后，若破坏了平衡，同样需要进行旋转操作来恢复平衡。
- 查找：与二叉排序树相同。

**时间复杂度：**

- 最好情况： $O(\log n)$
- 最坏情况： $O(\log n)$
- 平均情况： $O(\log n)$

### 1.3 哈希表

哈希表是一种通过哈希函数将关键字映射到表中一个位置的数据结构，以实现快速查找。哈希表的查找时间复杂度可以接近  $O(1)$ 。

**哈希函数**

哈希函数是将任意长度的数据映射为固定长度的数据。一个好的哈希函数应该能够尽可能均匀地分布关键字，减少冲突。

**处理冲突的方法**

- 链地址法：每个哈希表的槽位存储一个链表，所有哈希到同一槽位的关键字都存储在该链表中。
- 开放地址法：当发生冲突时，寻找下一个空槽位进行存储。常见的开放地址法有线性探测、二次探测、双重哈希等。

**时间复杂度：**

- 最好情况： $O(1)$ （没有冲突）
- 最坏情况： $O(n)$ （所有关键字都哈希到同一个槽位）
- 平均情况： $O(1)$ （假设哈希函数均匀分布关键字）

## 2. 实验内容

### 2.1 和有限的最长子序列

#### 2.1.1 问题描述

给你一个长度为  $n$  的整数数组 `nums` 和一个长度为  $m$  的整数数组 `queries`, 返回一个长度为  $m$  的数组 `answer`, 其中 `answer[i]` 是 `nums` 中元素之和小于等于 `queries[i]` 的子序列的最大长度。子序列是由一个数组删除某些元素（也可以不删除）但不改变元素顺序得到的一个数组。

#### 2.1.2 基本要求

##### 输入

第一行包括两个整数  $n$  和  $m$ , 分别表示数组 `nums` 和 `queries` 的长度

第二行包括  $n$  个整数, 为数组 `nums` 中元素

第三行包含  $m$  个整数, 为数组 `queries` 中元素

对于所有数据, 有  $1 \leq n, m \leq 1000$ ,  $1 \leq \text{nums}[i], \text{queries}[i] \leq 10^6$

##### 输出

输出一行, 包括  $m$  个整数, 为 `answer` 中元素

#### 2.1.3 数据结构设计

```
vector<int> nums(n);    // 数组 nums 的元素
vector<int> queries(m); // 数组 queries 的元素
```

#### 2.1.4 功能说明（函数、类）

```
class Solution {
public:
    // 输入数组 nums 和查询数组 queries, 返回查询数组 queries 的答案
    vector<int> maxSubsequence( vector<int>& nums, vector<int>& queries) {
        int n = nums.size();
        int m = queries.size();
        sort(nums.begin(), nums.end()); // 排序 nums
        vector<int> sum(n, 0);           // 数组 sum, 记录 nums 的前缀和
        sum[0] = nums[0];
        for (int i = 1; i < n; i++) // 计算前缀和
            sum[i] = sum[i - 1] + nums[i];
        vector<int> res(m, -1); // 数组 res, 记录查询数组 queries 的答案
        for (int i = 0; i < m; i++) {
            int l = 0, r = n - 1, mid;
            while (l <= r) { // 二分查找
                mid = (l + r) / 2;
                if (sum[mid] == queries[i]) { // 找到 queries[i] 的位置
                    res[i] = mid + 1; // 记录答案
                    break;
                } else if (sum[mid] < queries[i]) { // 右半边

```

```

        l = mid + 1;
    } else { // 左半边
        r = mid - 1;
    }
}
if (res[i] == -1)
    res[i] = 1; // 若 queries[i]不在 nums 中，则返回 nums 的最小位置
}
return res;
}
};

```

### 2.1.5 调试分析（遇到的问题和解决方法）

**遇到的问题：**在使用二分法查找最长子序列时，一开始我没有加入 `sum[mid] == queries[i]` 这个判断，仅仅只有小于和 `else`。这在判断顺序表中是否有待查找元素时是没问题的，但是本题需要返回小于等于待查找元素的最大值。如果待查找元素不再表中，在右边界处会出现错误处理的情况。

**解决方法：**二分查找中分别判断大于、小于和等于。这样无论待查找元素在不在表中都可以得到正确答案。

### 2.1.6 总结和体会

本题是最基础的顺序查找题目，难度很低。主要有一点值得注意：在对原数组排完序之后，直接先计算好所有的前缀和并存在一个数组中。这样在反复比较查询数组 `queries` 时就不需要一遍一遍的累加，降低了时间复杂度，提高程序运行效率。

## 2.2 二叉排序树

### 2.2.1 问题描述

二叉排序树 BST（二叉查找树）是一种动态查找表，基本操作集包括：创建、查找，插入，删除，查找最大值，查找最小值等。

本题实现一个维护整数集合（允许有重复关键字）的 BST，并具有以下功能：1. 插入一个整数 2. 删除一个整数 3. 查询某个整数有多少个 4. 查询最小值 5. 查询某个数字的前驱（集合中比该数字小的最大值）。

### 2.2.2 基本要求

#### 输入

第 1 行一个整数 `n`，表示操作的个数；

接下来 `n` 行，每行一个操作，第一个数字 `op` 表示操作种类：

若 `op=1`，后面跟着一个整数 `x`，表示插入数字 `x`

若 `op=2`，后面跟着一个整数 `x`，表示删除数字 `x`（若存在则删除，否则输出 `None`，若有多个则只删除一个），

若 `op=3`，后面跟着一个整数 `x`，输出数字 `x` 在集合中有多少个（若 `x` 不在集合中则输出 0）

若 `op=4`，输出集合中的最小值（保证集合非空）

若 op=5, 后面跟着一个整数 x, 输出 x 的前驱 (若不存在前驱则输出 None, x 不一定在集合中)  
输出

一个操作输出 1 行 (除了插入操作没有输出)

### 2.2.3 数据结构设计

```
struct TreeNode {
    int val; // 节点值

    int num; // 相同值节点的数量

    TreeNode* left; // 左子树指针

    TreeNode* right; // 右子树指针

    TreeNode(int x) : val(x), num(1), left(NULL), right(NULL) {} // 构造函数
};
```

### 2.2.4 功能说明 (函数、类)

```
bool delete_node(TreeNode*& p);

int Delete_BST(TreeNode*& T, int val) { // 删除节点

    if (T == nullptr) return false; // 待删除节点不存在, 返回 false

    if (val == T->val) return delete_node(T); // 找到待删除节点

    if (val < T->val) // 如果待删除节点在左子树中

        return Delete_BST(T->left, val); // 则递归左子树

    else // 如果待删除节点在右子树中

        return Delete_BST(T->right, val); // 则递归右子树

}

bool delete_node(TreeNode*& p) {

    if (p->num > 1) { // 待删除节点有多个相同值节点

        p->num--;

        return true;

    }

    TreeNode *q, *s;

    if (p->right == nullptr) { // 待删除节点没有右子树, 将左子树直接挂到待删除节点的位置

        q = p;

        p = p->left;

        delete q;

    } else if (p->left == nullptr) { // 待删除节点没有左子树, 将右子树直接挂到待删除节点的位置

        q = p;

        p = p->right;

        delete q;

    } else { // 待删除节点有左右子树, 找到左子树中的最大值, 替换待删除节点的值, 并删除最大值节点

        q = p;

        s = p->left;

        while (s->right != nullptr) { // 找到左子树中的最大值

            q = s;

            s = s->right;

        }

    }
```

```

    p->val = s->val; // 替换待删除节点的值

    p->num = s->num;

    if (q == p)
        q->left = s->left;
    else
        q->right = s->left;

    delete s;
}

return true;
}

```

```

int Get_Pre(TreeNode* T, int val) {
    if (T == nullptr) return -1; // 检查树是否为空
    TreeNode *p = T, *predecessor = nullptr;
    while (p) {
        if (p->val == val) {
            // 找到了节点
            if (p->left != nullptr) {
                // 如果左子树存在，找到左子树中的最大值
                predecessor = p->left;
                while (predecessor->right != nullptr) {
                    predecessor = predecessor->right;
                }
                return predecessor->val;
            } else {
                // 如果没有左子树，返回前一个节点
                return predecessor ? predecessor->val : -1;
            }
        } else if (val < p->val) {
            p = p->left;
        } else {
            // 更新前驱节点
            predecessor = p;
            p = p->right;
        }
    }
    // 如果没有找到，返回-1
    return predecessor ? predecessor->val : -1;
}

```

### 2.2.5 调试分析（遇到的问题 and 解决方法）

本题遇到的最大问题就是在删除节点的操作中，将小于被删除节点的最大节点替换到删除位置时，没有将结构体中的 num（即相同值的数量）一起替换过去，导致产生了一系列问题。最要命的是这个问题在测试数据基数较小时根本体现不出来，所以找了很长时间的 bug。。。

还有一个问题就是获取前置节点的函数中，predecessor 也就是前驱节点的更新原则。在做这题之前我一直认为前驱节点的作用就是存住上一次访问的节点便于回溯。但是经过这次 predecessor 的经验，我发现前驱节点的更新原则实际上是是否对查找逻辑起到贡献作用。这么说可能有点抽象。具体到这一题，predecessor 只有在对本题逻辑有贡献的情况下（也就是找到的节点比原先的前驱更大时）才要更新。

## 2.2.6 总结和体会

本题对于熟悉二叉排序树这一数据结构是一个非常好的题目，涉及到二叉排序树的诸多操作例如动态插入、删除节点、查找、获取最值以及获取前置值等操作。做完题目确实对二叉排序树以及各种操作的代码实现有了更深刻的了解和掌握。

另外我对是否需要更新前驱节点的原则也有了新的认识（我自己称之为贡献率），或许与组合数学证明容斥原理中提到贡献有关哈哈哈

## 2.3 换座位

### 2.3.1 问题描述

期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。

### 2.3.2 基本要求

期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。

### 2.3.3 数据结构设计

```
std::vector<std::vector<std::string>> old_chart(n, std::vector<std::string>(m)); //原先的座位表
std::vector<std::vector<std::string>> new_chart(n, std::vector<std::string>(m)); //目标座位表
```

### 2.3.4 功能说明（函数、类）

```
class Solution {
public:
    int solve(std::vector<vector<std::string>> &old_chart, std::vector<std::vector<std::string>> &new_chart) {
        int n = old_chart.size();
        int m = old_chart[0].size();
        int res = 0;
        unordered_map<string, pair<int, int>> target_pos; //记录每位同学的目标座位
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                target_pos[new_chart[i][j]] = make_pair(i, j); //记录目标座位
            }
        }
        for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < m; j++) {
            while (old_chart[i][j] != new_chart[i][j]) { //如果当前同学的位置与目标位置不同
                pair<int, int> pos = target_pos[old_chart[i][j]]; //记录当前位置同学的目标座位
                int x = pos.first;
                int y = pos.second;
                //将当前同学换到目标座位
                string temp = old_chart[i][j];
                old_chart[i][j] = old_chart[x][y];
                old_chart[x][y] = temp;
                res++; //换座位次数+1
            }
        }
    }
    return res;
}
};

```

### 2.3.5 调试分析（遇到的问题 and 解决方法）

本题没有遇到什么大问题，唯一需要注意的是因为已经有目标位置，所以直接建立元素和目标位置的映射，然后将不匹配的元素换到目标位置即可获取最小交换次数。

### 2.3.6 总结和体会

本题的交换方法像是一种提前知道目标位置的选择排序，因此不需要比较便可以直接交换，同时有着最小的交换次数。但也因此需要提前知道映射关系才能实现。

## 2.4 家族树

### 2.4.1 问题描述

人类学教授对居住在孤岛上的人们及其历史感兴趣。他收集了他们的家谱进行一些人类学实验。为了实验，他需要用电脑来处理家谱。为此，他把它们翻译成文本文件。以下是表示家谱的文本文件的示例。

```

John
  Robert
    Frank
    Andrew
  Nancy
    David

```

每行包含一个人的名字。第一行的名字是这个家谱中最古老的祖先。家谱只包含该祖先的后代。他们的丈夫或妻子没有在家庭树上显示。一个人的孩子比父母缩进一个空格。例如，Robert 和 Nancy 是 John 的孩子，Frank 和 Andrew 是 Robert 的孩子。David 比 Robert 缩进了一个空格，但他不是 Robert 的孩子，而是 Nancy 的孩子。为了用这一方法表示一个家谱，教授将一些人从家谱中排除，使得家谱中没有人有两个父母。

为了实验，教授还收集了家属的文件，并提取了每个家谱中关于两人关系的陈述语句。以下是关

于上述家庭陈述语句的一些例子。

John is the parent of Robert.

Robert is a sibling of Nancy.

David is a descendant of Robert.

对于实验，他需要判断每个陈述语句是否正确。例如，上面的前两个语句是正确的，最后一个语句是错误的。但是这个任务是十分无聊的，他想通过电脑程序来判断。

需要支持的查询有以下几种：

X is a child of Y.

X is the parent of Y.

X is a sibling of Y.

X is a descendant of Y.

X is an ancestor of Y.

注意：一个人的祖先、后代、兄弟可以是自己，但父母、孩子则不行。

人类学教授对居住在孤岛上的人们及其历史感兴趣。他收集了他们的家谱进行一些人类学实验。为了实验，他需要用电脑来处理家谱。为此，他把它们翻译成文本文件。以下是表示家谱的文本文件的示例。

John

Robert

Frank

Andrew

Nancy

David

每行包含一个人的名字。第一行的名字是这个家谱中最古老的祖先。家谱只包含该祖先的后代。他们的丈夫或妻子没有在家庭树上显示。一个人的孩子比父母缩进一个空格。例如，Robert 和 Nancy 是 John 的孩子，Frank 和 Andrew 是 Robert 的孩子。David 比 Robert 缩进了一个空格，但他不是 Robert 的孩子，而是 Nancy 的孩子。为了用这一方法表示一个家谱，教授将一些人从家谱中排除，使得家谱中没有人有两个父母。

为了实验，教授还收集了家属的文件，并提取了每个家谱中关于两人关系的陈述语句。以下是关于上述家庭陈述语句的一些例子。

John is the parent of Robert.

Robert is a sibling of Nancy.

David is a descendant of Robert.

对于实验，他需要判断每个陈述语句是否正确。例如，上面的前两个语句是正确的，最后一个语句是错误的。但是这个任务是十分无聊的，他想通过电脑程序来判断。

需要支持的查询有以下几种：

X is a child of Y.

X is the parent of Y.

X is a sibling of Y.

X is a descendant of Y.

X is an ancestor of Y.

注意：一个人的祖先、后代、兄弟可以是自己，但父母、孩子则不行。

## 2.4.2 基本要求

输入

输入包含若干组测试用例，每个测试用例由一个家谱和一个陈述句集合组成。

每个测试用例的第一行给出两个整数  $n, m$  ( $0 < n, m < 1000$ )，分别表示家谱中的名字和陈述语句的数目。

接下来输入的每行少于 70 个字符。名字的字符串仅由不超过 20 个字母字符组成。在家谱的第一行中给出名字前没有前导空格，在家谱中其他的名字至少缩进一个空格，表示是第一行给出的那个名字（家族的最早祖先）的后代。若家谱中一个名字缩进  $k$  个空格，则下一行最多缩进  $k+1$  个空格。

本题假定，除了最早的祖先外，在家谱中，每个人都有其父母。在同一个家谱中同样的名字不会出现两次。家谱的每一行结束的时候无多余的空格。

再接下来每句陈述句占一行。在家谱中没有出现的名字不会出现在陈述句中。陈述句中连续的单词被一个空格分开。每句陈述句在行首和行尾没有多余的空格。

用两个 0 表示输入的结束。

对于 100% 的数据，有  $0 \leq n \leq 5000$ ,  $0 \leq m \leq 10000$ ，且超过一个数据集

本题拿满分需要使用哈希表，可以使用 `std::map`

### 输出

对于测试用例中的每句陈述语句，程序输出一行，给出 True 或 False。

每个测试用例后要给出一个空行，就算只有一个测试用例。

## 2.4.3 数据结构设计

```
map<string, string> parentMap; // parentMap[x] = parent of x
```

## 2.4.4 功能说明（函数、类）

```
//主函数输入部分，构建 parentMap 映射，值为键的直系父亲
string ancestor, last; // ancestor 记录最高祖先，last 记录上一个节点，用于构建 parentMap
int lastIndent = 0; // 记录上一个节点的缩进（辈分）
for (int i = 0; i < n; ++i) {
    string line;
    getline(cin, line);

    int currentIndent = 0; // 当前节点的缩进（辈分）
    while (line[currentIndent] == ' ') ++currentIndent;

    string name = line.substr(currentIndent);

    if (i == 0) { // 第一个节点，为最高祖先
        parentMap[name] = name; // 置祖先为自己，方便后续判断
        ancestor = name; // 记录最高祖先
    } else {
        string parent = last;
        for (int j = 0; j < lastIndent + 1 - currentIndent; j++)
            parent = parentMap[parent]; // 向上寻找祖先
        parentMap[name] = parent;
    }

    last = name;
    lastIndent = currentIndent;
}
```

```
// 判断 x 是否是 y 的祖先
bool isAncestor(const string& x, const string& y,
                const map<string, string>& parentMap) {
    string current = y;
    if (x == y) return true; // 自己是自己的祖先。。。
    while (current != parentMap.at(current)) { // 一直向上找 y 的祖先
        if (parentMap.at(current) == x) return true;
        current = parentMap.at(current);
    }
    return false;
}
```

## 2.4.5 调试分析（遇到的问题解决方法）

本题遇到的问题在于如何构建合适的映射使得快速判断两个元素的关系。通过分析题目要求，无论是 parent、child、ancestor、sibling 还是 descendant，都只需要通过元素的直系父亲即可判断。因此构建 parentMap 映射，键是人名，值是父亲的名字。

## 2.4.6 总结和体会

本题让我体会到了映射的高效，我尝试过使用树来存储家族关系，但是这样结构体中要加入很多数据比如父节点、孩子节点的数组等等，不仅查找的效率低，且判断逻辑比较复杂。但是通过映射和递归的方式，可以简单高效的判断各种关系，非常方便。

其实本题的映射关系用 unordered\_map 更合适，因为这个映射并不需要有序，同时含有很多的查找操作，unordered\_map 会更高效一点。

## 2.5 最大频率栈

### 2.5.1 问题描述

设计一个类似堆栈的数据结构，将元素推入堆栈，并从堆栈中弹出出现频率最高的元素。

实现 FreqStack 类：

FreqStack() 构造一个空的堆栈。

void push(int val) 将一个整数 val 压入栈顶。

int pop() 删除并返回堆栈中出现频率最高的元素。如果出现频率最高的元素不只一个，则移除并返回最接近栈顶的元素。

### 2.5.2 基本要求

#### 输入描述

第一行包含一个整数 n

接下来 n 行每行包含一个字符串（push 或 pop）表示一个操作，若操作为 push，则该行额外包含一个整数 val，表示压入堆栈的元素

对于 100% 的测试数据， $1 \leq n \leq 20000$ ， $0 \leq val \leq 10^9$ ，且当堆栈为空时不会输入 pop 操作

#### 输出描述

输出包含若干行，每有一个 pop 操作对应一行，为弹出堆栈的元素

#### 提交要求

本题需要提交类 FreqStack 的实现，类中需要包含三个名为 FreqStack、push、pop 的函数

### 2.5.3 数据结构设计

```
map<int, vector<int>> stack;           // 存放频率和栈的映射
unordered_map<int, int> freqCount;    // 存放频率和频率的映射
```

### 2.5.4 功能说明（函数、类）

```
void push(int val) {
    freqCount[val]++; //压入的元素对应的频率+1
    stack[freqCount[val]].push_back(val); //将元素压入对应频率的栈中
}

int pop() {
    int ans = (--stack.end())->second.back(); //取出最大频率的栈顶元素
    stack[freqCount[ans]].pop_back(); //弹出栈顶元素
    if (stack[freqCount[ans]].empty()) { //如果栈为空，则删除该频率的栈
        stack.erase(freqCount[ans]);
    }
    freqCount[ans]--; //弹出的元素对应的频率-1
    return ans;
}
```

### 2.5.5 调试分析（遇到的问题解决方法）

本题遇到的问题是获取最大频率的元素，以及频率相同时弹出最近压入的值。后者很好解决，只需维护相同频率对应的一个栈即可。因此考虑创建频率和栈的映射 stack。因为 map 是按照键的值升序排列，所以可以很方便的获取最大频率对应的栈。同时维护一个值和频率的对应关系用于记录值的个数（其实用数组也可以，但是映射更高效）

### 2.5.6 总结和体会

本题主要体会 map 和 unordered\_map 的区别。两者都是用于存储键值对的关联容器。Map 的实现基于红黑树，键值对按照键的顺序自动排序。其插入、删除、查找、访问的时间复杂度均为  $O(\log n)$ 。unordered\_map 的实现基于哈希表，键值对不保证任何特定的顺序。其插入、删除、查找、访问的平均时间复杂度为  $O(1)$ ，最坏时间复杂度均为  $O(n)$ 。

map 适用于需要有序键值对的场景。适用于需要快速查找、插入和删除操作，并且需要保持键的有序性。unordered\_map 适用于需要快速查找、插入和删除操作，并且不需要保持键的有序性。

### 3. 实验总结

在本次实验中,我深入学习了多种数据结构及其相关算法,涵盖了静态查找表、动态查找表、哈希表、二叉排序树、频率栈等。

静态查找表:

学习了顺序查找、二分查找和分块查找的实现及其时间复杂度。

动态查找表:

通过实现二叉排序树(BST)和AVL树,深入理解了树的基本操作,包括插入、删除、查找等。特别是AVL树的平衡操作让我意识到数据结构的平衡性对查找效率的影响。

在实现过程中,遇到的问题主要集中在节点删除和前驱节点的查找上,通过调试和反复测试,逐步掌握了这些操作的细节。

哈希表:

学习了哈希函数及冲突处理的方法,特别是链地址法和开放地址法的实现。通过哈希表的应用,感受到了 $O(1)$ 查找的高效性。

在实现过程中,注意到哈希函数的设计对性能的影响,良好的哈希函数能够有效减少冲突,提高查找效率。

通过实现频率栈,掌握了如何在栈中管理元素的频率,并能够在 $O(1)$ 时间内弹出频率最高的元素。这一题目也让我认识到如何结合多种数据结构(如栈和哈希表)来解决复杂问题。

在处理家族树的题目时,学习了如何使用映射来快速判断两个人之间的关系。通过构建父子关系的映射,能够高效地回答关于祖先、后代和兄弟关系的问题。这一过程让我体会到映射的高效性,相比于树结构,映射在查找和判断逻辑上更为简洁明了。

同时,每个题目的实现都让我在实践中巩固了理论知识。例如,在处理“换座位”问题时,通过建立元素与目标位置的映射,能够高效地计算最小交换次数;而在“最大频率栈”中,通过维护频率与栈的映射,能够快速实现频率最高元素的弹出。这些实现不仅提升了我的编程能力,也让我对算法的设计思路有了更深的理解。

每个题目的实现中,我都对时间复杂度进行了分析,理解了不同算法在不同情况下的表现。

总结

通过不同题目的实践,我认识到选择合适的数据结构对于提高算法效率的重要性。本次实验让我对查找相关的数据结构和算法有了更全面的理解与掌握。通过实践,我不仅提高了编程能力,也增强了分析和解决问题的能力。